

GRIMOIRE VAULT · V1.0.0

Разработчик

*Гайд разработчика — стек, локальный запуск,
расширение*

Grimoire Vault — гайд разработчика

Для тех, кто хочет понять, как это устроено внутри, поднять локально, внести изменения или зафоркнуть под себя.

1. Tech stack за 30 секунд

СЛОЙ	ТЕХНОЛОГИЯ	ПОЧЕМУ
Frontend	Next.js 16 (App Router, RSC, Turbopack) + React 19	Server Components сокращают bundle, RSC + Suspense streaming для быстрого первого пейнта
UI	Tailwind v4 + Fraunces / Manrope / JetBrains Mono	Единый design system без отдельной CSS-сборки
Auth + DB	Supabase (Postgres + Auth + Realtime + RLS)	Бесплатный tier с 500 MB Postgres, RLS = per-row авторизация без своего бэкенда
Search	Postgres <code>tsvector</code> (Russian) + pgvector (HNSW, 384-dim)	FTS + cosine similarity в одной БД, без отдельного Elastic/Pinecone
Embeddings	<code>@huggingface/transformers</code> + <code>multilingual-e5-small</code>	Считаются в браузере , ноль API-ключей
Files	Cloudflare R2 (\$0 egress) с presigned PUT	Отделение бинарников от Postgres
Bot	grammY + webhook на Vercel	Один TS-файл, type-safe
Crypto	Web Crypto API (PBKDF2 + AES-GCM)	В браузере, master password не уходит на сервер
Hosting	Vercel (serverless, edge для health, Node для тяжёлых)	Бесплатный tier с 60-сек таймаутом
Cron	Vercel Cron Jobs	<code>/api/telegram/digest</code> в 06:00 UTC
PWA	Manual service-worker	Без зависимостей типа Workbox
Push	Web Push (VAPID)	Нативные уведомления на iOS PWA / Android / Desktop
Rate limit	Upstash Redis (с in-memory fallback)	Sliding window per-user, без своих infra
Observability	Vercel Logs + structured JSON	<code>level / route / durationMs / requestId</code>

Полный roadmap того, как мы дошли до этого стека — в `PROJECT-STORY.md`.

2. Локальный запуск

2.1 Prerequisites

- **Node 22+** (мы тестируем на 24)
- Аккаунт на Supabase (бесплатный)
- Аккаунт на Cloudflare R2 (бесплатный до 10 GB)
- Telegram-бот через @BotFather (опционально)

2.2 Установка

```
git clone <this-repo>
cd grimoire-vault
npm install
cp .env.example .env.local
# заполни .env.local своими значениями (см. раздел 3)
```

2.3 Schema

В Supabase Dashboard → SQL Editor выполни миграции в порядке:

```
supabase/migrations/20260504000000_initial_schema.sql
supabase/migrations/20260504010000_credentials_per_field_iv.sql
supabase/migrations/20260504020000_count_entries_per_category.sql
supabase/migrations/20260504030000_embedding_384.sql
supabase/migrations/20260504040000_entries_triangled_at.sql
supabase/migrations/20260504050000_dedup_index_unpartial.sql
supabase/migrations/20260504060000_push_subscriptions.sql
supabase/migrations/20260504070000_shared_vaults.sql
```

После этого вызови `vault_members_select_self` policy fix (в проде дропали и пересоздавали — см. CHANGELOG):

```
drop policy if exists "vault_members_select_self" on public.vault_members;
create policy "vault_members_select_self" on public.vault_members
  for select using (auth.uid() = user_id);
```

И `SECURITY DEFINER` на trigger:

```
alter function public.vaults_seed_owner() security definer;
```

*Лучше: оба фикса встроены в `20260504070000_shared_vaults.sql` для новых установок.
Эти ad-hoc патчи понадобились только проде из-за порядка применения.*

2.4 R2 bucket

Создай bucket (например, `baza`) → выпусти R2 API token (`Object Read & Write`) → пропиши в `.env.local`.

CORS-правила применяются скриптом:

```
node scripts/r2-setup.mjs
```

2.5 Auth provider в Supabase

Authentication → Providers → Email:

- Site URL: `http://localhost:3000` (и продакшн-домен)
- Redirect URLs: `http://localhost:3000/auth/callback` + прод

2.6 Запуск

```
npm run dev
# → http://localhost:3000
```

3. Environment variables — полный список

Обязательные

```
NEXT_PUBLIC_SUPABASE_URL=https://YOUR_PROJECT.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cGE6YWV0Ij...
```

Server-only — бот и cron используют, browser — никогда.
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIsInR5cGE6YWV0Ij...

CLOUDFLARE_R2_ACCOUNT_ID=...
CLOUDFLARE_R2_ACCESS_KEY_ID=...
CLOUDFLARE_R2_SECRET_ACCESS_KEY=...
CLOUDFLARE_R2_ENDPOINT=https://<account>.r2.cloudflarestorage.com
CLOUDFLARE_R2_BUCKET=baza

NEXT_PUBLIC_APP_URL=http://localhost:3000 # или продакшн URL

Опциональные

```
# Telegram bot — нужен, если хочешь пользоваться ботом.
TELEGRAM_BOT_TOKEN=...
TELEGRAM_WEBHOOK_SECRET=любой-длинный-секрет-32+
```

Owner-only routes (admin stats / health / wipe). Без неё — fail-closed.
OWNER_EMAIL=you@example.com

Web Push — нужен для уведомлений. Сгенерировать:
node -e "console.log(require('web-push').generateVAPIDKeys())"
VAPID_PUBLIC_KEY=...
VAPID_PRIVATE_KEY=...
VAPID_SUBJECT=mailto:you@example.com
NEXT_PUBLIC_VAPID_PUBLIC_KEY=<тот же что VAPID_PUBLIC_KEY>

Upstash Redis — для распределённого rate limit'a. Без — fallback на in-memory.
UPSTASH_REDIS_REST_URL=https://YOUR-DB.upstash.io
UPSTASH_REDIS_REST_TOKEN=AYAAA...

Direct Postgres pooler (Supervisor 6543) — пока никем не используется,
документировано на будущее.
SUPABASE_DB_POOLER_URL=postgres://postgres.YOUR_PROJECT:PASSWORD@aws-0-
REGION.pooler.supabase.com:6543/postgres

4.1 Папки

```
grimoire-vault/
├─ app/
│  ├─ (app)/                # auth-protected pages
│  │  ├─ layout.tsx         # Header + Footer + auth-guard + ⚡K + IdlePreload
│  │  ├─ page.tsx          # Home – featured + recent + categories grid
│  │  ├─ categories/       # Все 13 в сетке
│  │  ├─ category/[id]/    # CategoryView с keyboard nav + bulk-select
│  │  ├─ inbox/            # Триаж UI
│  │  ├─ search/           # FTS / Hybrid / Semantic + bulk-select
│  │  ├─ kanban/           # @dnd-kit board
│  │  ├─ settings/         # account, telegram, push, vaults, export, admin
│  │  ├─ invite/[code]/    # Accept invite landing
│  │  └─ admin/health/     # Owner-only dependency probe
│  ├─ (auth)/login/        # magic-link + password
│  ├─ auth/callback/       # OAuth code exchange
│  └─ api/                 # 25+ route handlers
├─ components/             # shared React components
├─ lib/                   # data layer, hooks, helpers
├─ supabase/migrations/   # idempotent SQL (8 files)
├─ scripts/               # E2E suites + setup
├─ tests/e2e/             # Playwright specs
├─ public/                # PWA manifest, sw.js, icons
└─ docs/                  # ← вы здесь
```

4.2 Слои в lib/

```
lib/
├─ supabase/              # client / server / middleware Supabase factories
├─ data/                  # entries, kanban, credentials, vaults, search, telegram
├─ schemas/               # Zod схемы для всех API routes
├─ hooks/                 # useEntries, useKanban, useVaults, useEntryKeyboardNav, ...
├─ embeddings/            # browser-side e5-small pipeline
├─ telegram/bot.ts        # grammY-based handlers
├─ crypto.ts              # PBKDF2 + AES-GCM
├─ r2.ts                  # S3 client + presign + list/get/batch-delete
├─ og.ts                  # SSRF-safe og: meta extractor
├─ dedup.ts               # URL canonicalize + sha256 → content_hash
├─ upload.ts              # browser → R2 with WebP transcode
├─ push.ts                # VAPID + send-to-user
├─ ratelimit.ts           # Upstash | in-memory token bucket
├─ log.ts                 # structured JSON logger
├─ admin.ts               # requireOwner / isOwnerEmail
├─ api-helpers.ts         # withErrorHandler + requireUser + parseBody + timing
├─ api-client.ts          # typed fetch wrappers (entriesApi, searchApi, ...)
├─ errors.ts              # DataError with extra payload
├─ utils.ts               # humanSize и пр.
├─ categories.ts          # registry 13 категорий
└─ types/                 # shared TS types
```

4.3 Слой пагинации запросов

Каждый API route:

- Идёт через `withErrorHandler` (`lib/api-helpers.ts`)
 - Generates `crypto.randomUUID()` per request
 - Times the call (start to finish)

- Logs structured JSON to Vercel: `{level, route, method, status, durationMs, requestId, ...}`

GRIMOIRE VAULT · DEVELOPER

- On error: response body + `X-Request-Id` header carry the UUID

2. Вызывает `requireUser()` или `requireOwner()` для auth
3. Опционально вызывает `checkRateLimit(userId, scope, profile)`
4. Парсит body через Zod (`parseBody`)
5. Делегирует в `lib/data/...` который работает с RLS-scoped Supabase client

4.4 RLS (Row-Level Security) — основа модели

В Postgres каждая user-таблица имеет policies:

```
-- Пример: entries
create policy "entries_select_member" on public.entries
for select using (
  (vault_id is null and user_id = auth.uid())           -- personal
  or
  (vault_id is not null and exists (                    -- shared
    select 1 from public.vault_members vm
    where vm.vault_id = entries.vault_id and vm.user_id = auth.uid()
  ))
);
```

`auth.uid()` приходит из JWT, который Supabase SSR-клиент кладёт в куку `sb-<project_ref>-auth-token`. PostgREST читает JWT и сетит `auth.uid()` для каждого SQL-запроса.

Service-role client (`createServiceClient()` в `lib/supabase/server.ts`) обходит RLS — используется для:

- Telegram webhook (нет user-cookie)
- Cron jobs
- Admin endpoints (auth-проверка делается отдельно)
- Импорт (cross-table inserts)

4.5 Realtime

Хуки (`useEntries`, `useKanban`, `useCredentials`, `InboxBadge`, `useVaults`) подписываются на `postgres_changes` через Supabase Realtime. RLS применяется ко всем событиям → пользователь видит только свои изменения.

```
const channel = supabase
  .channel(`entries:${categoryId}`)
  .on("postgres_changes", { event: "*", schema: "public", table: "entries", filter:
    `category_id=eq.${categoryId}` }, handler)
  .subscribe();
```

5. Schema migrations

Все миграции — idempotent (используют `if not exists`, `drop ... if exists`).

#	ФАЙЛ	ЧТО ДОБАВЛЯЕТ
1	2026050400000_initial_schema.sql	7 таблиц, 18 RLS policies, 8 индексов, realtime publication, 13 категорий
2	20260504010000_credentials_per_field_iv.sql	Per-field IVs для AES-GCM (одна IV на запись = недостаточно)
3	20260504020000_count_entries_per_category.sql	RPC count_entries_per_category() — server-side aggregation для Home
4	20260504030000_embedding_384.sql	vector(1536) → vector(384) + HNSW индекс + search_entries_semantic RPC
5	20260504040000_entries_triangled_at.sql	entries.triangled_at + partial index + auto-trigger для не-bot записей
6	20260504050000_dedup_index_unpartial.sql	Дроп WHERE clause на dedup-индексе для PostgREST upsert ON CONFLICT
7	20260504060000_push_subscriptions.sql	Web Push subscription store
8	20260504070000_shared_vaults.sql	vaults + vault_members + vault_invites + entries.vault_id + new RLS

Применение

Через Supabase Management API:

```
PROJECT_REF=your-project-ref
TOKEN=sbp...

SQL=$(node -e "console.log(JSON.stringify({query:
require('fs').readFileSync('supabase/migrations/<name>.sql','utf8')})))"
curl -X POST "https://api.supabase.com/v1/projects/$PROJECT_REF/database/query" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d "$SQL"
```

Или Dashboard SQL Editor → Run.

6. API endpoints

CRUD entries

- GET /api/entries — list (filters: categoryId, pinned, tag, q, triage, importedVia, vaultId)
- POST /api/entries — create (server вычислит content_hash для dedup)
- GET /api/entries/[id] — get one
- PATCH /api/entries/[id] — partial update (поддерживает embedding, triagedAt, vaultId)
- DELETE /api/entries/[id]

Search

- GET /api/search?q=... — FTS
- POST /api/search body: {q, embedding[384], mode: "semantic"|"hybrid"} — cosine / RRF

Other

- `POST /api/extract` — server-side og: meta extractor (rate-limited)
- `GET /api/export`, `GET /api/export/full`, `POST /api/import`
- `GET/POST /api/vaults`, `GET/DELETE /api/vaults/[id]`, `GET/DELETE /api/vaults/[id]/members?user=<uuid>`, `GET/POST/DELETE /api/vaults/[id]/invites`, `POST /api/vault-invites/[code]`
- `POST/DELETE /api/push/subscribe`, `POST /api/push/test`
- `POST /api/r2/presign`, `GET /api/r2/object/[...key]`
- `POST /api/telegram` (webhook), `POST /api/telegram/digest` (cron), `POST /api/telegram/link-code`
- Owner-only: `GET /api/admin/stats`, `GET /api/admin/health`, `POST /api/admin/wipe`
- `GET /api/health` (public)

GRIMOIRE VAULT · DEVELOPER

7. Тестирование

TypeScript + ESLint

```
npx tsc --noEmit          # strict typecheck
npx eslint .              # lint
```

Production build

```
npx next build             # Turbopack
```

API verification — 70 проверок

```
ANON=$NEXT_PUBLIC_SUPABASE_ANON_KEY \
SERVICE=$SUPABASE_SERVICE_ROLE_KEY \
APP=https://grimoire-vault.vercel.app \
node scripts/backlog-verify.mjs
```

Покрывает auth gates, dedup, export/import round-trip, rate limiting, duplicates, triage, hybrid search, Phase 6 embeddings, og: extract.

Browser E2E — 15 Playwright тестов

```
ANON=... SERVICE=... npx playwright test
```

Покрывает Recent entries, Inbox badge live update, 🗂 K palette, shift+click bulk, j/k nav, ? help, localStorage round-trip, hotkey suppression в input'ax, /admin/health page gate.

Backend integration scripts

```
APP_BASE=https://grimoire-vault.vercel.app \
ANON=... SERVICE=... \
TELEGRAM_WEBHOOK_SECRET=... \
node scripts/e2e-telegram.mjs          # bot end-to-end
node scripts/e2e-r2-upload.mjs         # presigned upload + RLS isolation
node scripts/e2e-credentials.mjs       # AES-GCM round-trip
node scripts/e2e-phase5.mjs            # search + edit + kanban DnD
```


Vercel

```
vercel link --project grimoire-vault

# Push every env var to production:
for line in $(grep -v '^#' .env.local | grep -v '^$'); do
  KEY="${line%%=*}"; VAL="${line#*=}"
  printf "%s" "$VAL" | vercel env add "$KEY" production --force
done

vercel deploy --prod --yes
```

Telegram webhook

После деплоя:

```
curl -X POST "https://api.telegram.org/bot<TOKEN>/setWebhook" \
-d "url=https://<your-domain>/api/telegram" \
-d "secret_token=<TELEGRAM_WEBHOOK_SECRET>" \
-d "allowed_updates=[\"message\"]" \
-d "drop_pending_updates=true"
```

Cron

`vercel.json` объявляет digest в `0 6 * * *` UTC. Vercel автоматически регистрирует cron при деплое.

9. Расширение (как добавить...)

9.1 Новую категорию

Не делай этого, если не понимаешь зачем — 13 «комнат» — фиксированная часть продуктового дизайна. Тебе скорее нужен тег, не категория.

Если всё-таки:

1. `lib/types/index.ts` — добавь в `CategoryId` union.
2. `lib/categories.ts` — добавь объект с `no, en, ru, icon, ordering`.
3. SQL: `INSERT INTO public.categories ...` (вручную в Supabase или новой миграцией).
4. Создай иконку в `components/icons/Icon.tsx` если ещё нет.
5. Добавь любую специальную UI-логику в `CategoryView` (например, как у Video / Media категорий).
6. Обнови Zod-enumy в `lib/schemas/entries.ts`.

9.2 Новое поле на Entry

1. SQL миграция: `ALTER TABLE entries ADD COLUMN ...`
2. `lib/types/index.ts` — добавь в `Entry`.
3. `lib/data/mappers.ts` — обнови `rowToEntry` и `entryToRow` (round-trip).
4. `lib/schemas/entries.ts` — обнови `createEntrySchema` + `updateEntrySchema`.

5. UI: добавь поле в `AddItemModal` / `EditEntryModal`.

6. **Не забудь**: если поле имеет default value, не добавляй его в

GRIMOIRE VAULT · DEVELOPER

`createEntrySchema.default(...)` — иначе сломается partial PATCH через

`updateEntrySchema`. См. CHANGELOG про PATCH-default bug.

9.3 Новый rate-limit profile

`lib/ratelimit.ts` → `RATE_LIMITS`. Затем в роуте:

```
const limited = await checkRateLimit(user.id, "my-scope", RATE_LIMITS.myScope);
if (limited) return limited;
```

9.4 Новую миграцию

```
# По соглашению — `YYYYMMDDHHMMSS_description.sql` в supabase/migrations/.
# Idempotent: используй IF NOT EXISTS / DROP IF EXISTS.
```

Применение через Management API или Dashboard SQL Editor.

9.5 Новый bot-command

`lib/telegram/bot.ts` → добавь `bot.command(...)`. Используй `safeReply` вместо `ctx.reply` для устойчивости (`chat_not_found` etc).

10. Code conventions

TypeScript

- Strict mode on, `noEmit` для проверки.
- Predefined types в `lib/types/index.ts` — domain language.
- Zod schemas в `lib/schemas/` — single source of truth для API.

Server vs Client

- `"use client"` только когда нужно (interactive components, browser APIs).
- `"server-only"` маркер в каждом DAL-модуле — гарантирует, что service-role и секреты не утекут на клиент.
- Server Components — async, делают DB-запросы прямо в JSX.
- Suspense boundaries для streaming тяжёлых частей.

Errors

- `DataError(message, status, extra?)` для domain errors из DAL.
- `HttpError(message, status, extra?)` для HTTP-specific (например, 401).
- `withErrorHandler` маппит обе на структурированные JSON responses с `requestId`.

Logging

- Через `lib/log.ts` (`log.error` / `log.warn` / `log.info`).
- Структурированный JSON, индексируется Vercel Logs Explorer.
- НЕ логируй: тела запросов, plaintext credentials, embeddings.

RLS

- ВСЕГДА предпочитай RLS-scoped client (`createClient()` в `server.ts`) service-role'y.

- Service-role — только когда RLS физически мешает (cross-user reads, bot inserts, admin endpoints). Каждый такой случай явно комментируется.

11. Performance

Bundle splitting

- `next/dynamic({ ssr: false })` для тяжёлого UI (KanbanBoard 18 KB, CredentialsView 25 KB).
- Lazy-loaded modals (AddItemModal, EditEntryModal) — не в initial bundle.
- IdlePreload (`components/layout/IdlePreload.tsx`) прогревает кэш лениво-загружаемых модулей через `requestIdleCallback` после первой интерактивной отрисовки.

Database

- Partial indexes на дорогих фильтрах (`entries_inbox_idx`, `entries_dedup_idx`).
- HNSW для cosine similarity (быстрее ivfflat на small/medium corpora).
- Server-side aggregations через RPC (`count_entries_per_category`) вместо стрима в JS.

R2 / files

- Browser-side WebP transcode в `lib/upload.ts` — экономит 30-60% трафика.
- Service-worker cache-first для static, SWR для images, network-first для pages.

Embeddings

- Считаются в браузере, не на сервере → ноль cold-start latency.
- Модель скачивается один раз в IndexedDB (~30 MB), потом мгновенно.

12. Security model

GRIMOIRE VAULT · DEVELOPER

CONCERN	MITIGATION
Cross-user data leakage	RLS на каждой user-таблице; service-role используется только в trusted server routes (bot, cron, admin) с явным <code>WHERE user_id = ...</code>
Credentials at rest	Browser-side AES-GCM-256, master pwd живёт в sessionStorage; сервер видит только ciphertext + per-field IVs
File hot-linking	R2 bucket приватный; downloads стримятся через <code>/api/r2/object/[...key]</code> с ownership-проверкой
Bot impersonation	Webhook верифицирует header <code>secret_token</code> (Telegram аппендит после <code>setWebhook</code>)
Cron impersonation	Тот же <code>secret_token</code> или <code>user-agent: vercel-cron</code>
XSS	Server-rendered, <code>dangerouslySetInnerHTML</code> только для sanitised search-snippets
CSRF	Same-origin cookies, no <code>Set-Cookie</code> cross-domain
SSRF в <code>/api/extract</code>	Блокирует loopback / private IPv4-IPv6 / non-http(s) перед fetch
Rate-limit bypass	Per-(userId, scope) bucket; auth check перед rate check
Owner-only routes	<code>OWNER_EMAIL</code> env-gate; fail-closed при отсутствии env-vars
Master password reset	Не предусмотрено by design. Если забыл — credentials потеряны. Сервер не может дешифровать.

13. Observability

Logs

Каждый API-вызов через `withErrorHandler` пишет одну JSON-строку:

```
{
  "level": "info" | "warn" | "error",
  "ts": "2026-05-04T18:00:00.000Z",
  "msg": "request" | "<error message>",
  "requestId": "<uuid>",
  "route": "/api/entries",
  "method": "GET",
  "status": 200,
  "durationMs": 142,
  "stack": "...",      // только на 5xx
  "issues": [...]     // только на ZodError
}
```

Уровни:

- 2xx/3xx → `info` (`msg: "request"`)
- 4xx → `warn` (без stack)
- 5xx → `error` (с stack)

Vercel Logs Explorer

GRIMOIRE VAULT · DEVELOPER

- Filter `level:error` → bugs только.
- Filter `route:/api/search` → per-route latency.
- Sort by `durationMs` desc → находить slow queries.
- Поиск по `requestId:<uuid>` → один запрос с полным контекстом.

X-Request-Id

Только на error responses. В JSON body тоже как `requestId`. Когда пользователь сообщает баг — это единственный grep-string для логов.

/admin/health

Owner-only страница — пробит каждой зависимости (Supabase REST, pgvector RPC, R2 bucket, Telegram getMe / getWebhookInfo) с round-trip latency. Запускать после каждого деплоя.

/admin/stats

Owner-only ops dashboard — счётчики, R2-разбивка, embedding coverage, last-bot-import. Refresh on demand.

14. Continuous integration

Пока CI не настроен (личный проект). Для serious-mode установи:

```
# .github/workflows/ci.yml – sketch
name: CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci
      - run: npx tsc --noEmit
      - run: npx eslint .
      - run: npx next build
      - run: npx playwright install --with-deps chromium
      - run: npx playwright test
    env:
      ANON: ${ secrets.SUPABASE_ANON_KEY }
      SERVICE: ${ secrets.SUPABASE_SERVICE_ROLE_KEY }
      BASE_URL: https://grimoire-vault-staging.vercel.app
```

15. Отладка типичных проблем

«Hydration mismatch» в DevTools

Проверь, не использует ли Server Component `Date.now()` или `Math.random()` без обертки. Чаще всего — Date в format render'e.

«infinite recursion detected in policy»

RLS policy ссылается на свою же таблицу через EXISTS. Сделай policy non-recursive (например, `auth.uid() = user_id` на base case).

Insert violates RLS на shared vaults

Trigger без `SECURITY DEFINER` пытается INSERT по cookie-сессии и упирается в WITH CHECK. Решение: добавь `SECURITY DEFINER` на trigger function ИЛИ используй service-role в DAL'е для INSERT'а.

`entries.embedding` is null после reindex

Запусти `Settings → Reindex embeddings` ещё раз. В прошлый раз могла быть quota / network issue. В Vercel Logs ищи warn'ы по `route:/api/entries/.../update`.

Service worker не обновляется

Bump `VERSION` constant в `public/sw.js`. Браузер скачает новую версию при следующей навигации. В DevTools → Application → Service Workers → “Update on reload” во время разработки.

16. Известные ограничения

- **Один Vercel-деплой = один Supabase-проект** в коде захардкожен `PROJECT_REF` в нескольких скриптах (`scripts/*.mjs`, `tests/e2e/auth-and-crud.spec.ts`). При форке надо заменить.
- **Embedding model — английский bias**. multilingual-e5-small на русском работает хуже, чем на английском. Запросы на смешанной лексике лучше всего.
- **Bot работает только с personal vault'ом**. Не клал в shared.
- **Credentials не шарятся**. Master password — личный.
- **Push на iOS только через установленный PWA (16.4+)**.
- **In-memory rate limiter** — across function instances не consistent. Включай Upstash, если нужны жёсткие лимиты.

17. Roadmap-кладбище

История того, что строилось — в `PROJECT-STORY.md`. Полный список фичей с описанием — в `CHANGELOG.md`.

18. Архитектурные паттерны последних волн (22–25)

Эти паттерны новые и стоит понимать прежде чем расширять соответствующий код.

18.1 ThemedSelect — общий тёмный дропдаун

`components/forms/ThemedSelect.tsx` — generic-замена нативного `<select>`. Принимает `options: SelectOption[]` и стандартный `value/onChange`. Используется в:

- `EditKanbanModal` (Колонка / Приоритет / Связь с категорией)
- `AddKanbanModal` (то же)

- `AddItemModal` / `EditEntryModal` для prompts (поле «Модель»)
- `CollectionSelect` использует тот же UX-паттерн (открытие, ESC, ArrowUp/Down/Enter, mousedown-select), но с депт-вложенностью

GRIMOIRE VAULT DEVELOPER

Не ходит через portal — рендерится `absolute` относительно обёртки. Если когда-нибудь понадобится в overflow-clip контексте — переписать на portal или фиксированное позиционирование.

18.2 Sort + tag pipeline в `CategoryView`

Композиция фильтров строго по порядку:

```
items → collectionFiltered → tagFiltered → sorted → pinned/others
```

- `collectionFiltered` — берёт `selectedCollection` из `CollectionsTabs`, использует BFS по `parentId`-дереву чтобы выбрать дескендантов (родительская коллекция включает суб-).
- `tagFiltered` — фильтр по `selectedTag`, активен только в режиме `sortMode === "tags"`.
- `sorted` — `useMemo([filtered, sortMode])` с пятью режимами, RU-aware `localeCompare("ru", { sensitivity: "base" })`.

Состояние сортировки персистится в localStorage пер-категория (`grimoire:sort:<categoryId>`). При смене режима с «tags» на любой другой — `selectedTag` сбрасывается автоматически.

18.3 Image compression pipeline

`lib/image-compress.ts` использует Canvas API без внешних зависимостей. Ключевые решения:

- `createImageBitmap` с `imageOrientation: "from-image"` — EXIF orientation применяется к битмапу до отрисовки. Старые браузеры без опции деградируют до raw orientation, не критично.
- **OffscreenCanvas с fallback на HTMLCanvasElement** — первый быстрее (можно мигрировать в Web Worker позже), второй для совсем старых браузеров.
- **Try WebP, fallback to JPEG** — некоторые Safari-сборки отказываются от `convertToBlob({ type: "image/webp" })` на OffscreenCanvas.
- **Always-recompress with size guard** — `compressImage()` всегда пытается re-encode raster, но если результат больше оригинала — возвращает оригинал. Net loss never.

API: `compressImage(file, { targetBytes?, maxDim?, quality?, onStep? })`. Без `targetBytes` — одна попытка с дефолтами. С `targetBytes` — адаптивная цепочка: quality 0.82 → 0.4, потом dim × 0.75 до floor 480px.

18.4 Kanban realtime — debounce + quiet window

В `lib/hooks/useKanban.ts` две защиты от каскада postgres_changes событий:

1. **Coalescing scheduler** (`REFETCH_DEBOUNCE_MS = 400`) — `scheduleRefetch()` использует один shared timer, перезапускает при каждом событии.
2. **Local-write quiet window** (`LOCAL_WRITE_QUIET_MS = 1500`) — `markLocalWrite()` ставит `localWriteUntil = Date.now() + 1500` перед каждой мутацией. Realtime handler игнорирует события если `Date.now() < localWriteUntil`.

Каждая мутация (`create`, `update`, `remove`, `moveCard`) обязана вызвать `markLocalWrite()` ПЕРЕД API-вызовом. Иначе echo-cascade прилетит и перезапишет оптимистичный апдейт

промежуточным состоянием.

`update()` теперь делает **полноценный optimistic patch** локального состояния (раньше был только API call) — необходимо потому что quiet window блокирует refetch. Без локального патча UI бы 1.5 с показывал старое состояние.

18.5 Custom Kanban columns без миграции

Паттерн «слагги фиксированы в БД, имена редактируемы локально»:

- `kanban_cards.column_name` остался `text` (был enum в Zod, теперь `[a-z0-9_-]{1,40}`). DB допускает любой слаг.
- Кастомные колонки — массив `{ slug, name }` в `localStorage` (`grimoire:kanban:custom-cols`).
- Имена дефолтных колонок — отдельная мапа в `localStorage` (`grimoire:kanban:default-names`), позволяет переименовать Backlog/Doing/Done без ломки внутренних слаггов.
- При `computeColumns: defaults` (с применённым `override`-именем) + `customColumns` + `orphan slugs` (найденные в `board[slug]` но отсутствующие в обоих списках, рендерятся со слагом как именем).

Trade-off: пустая custom-колонка не переживает `clear-storage` / другое устройство. Колонка с хотя бы одной карточкой — выживает через `column_name` в Postgres. Если когда-то понадобится cross-device parity — отдельная таблица `kanban_columns` (миграция готова в голове, не реализована).

18.6 ProjectPanel pattern

`components/entry/ProjectPanel.tsx` — пример «category-specific detail panel» на entry detail page. Условие в `app/(app)/entry/[id]/page.tsx`:

```
{entry.categoryId === "portfolio" && <ProjectPanel entry={entry} />}
```

Внутри:

- `entry.body` — T3 (textarea с debounced autosave 800 мс)
- `entry.metadata.vercelUrl / gitUrl / dbUrl` — quick links
- `entry.metadata.extraLinks: { label, url }[]` — custom links
- `entry.metadata.creds: { label, value }[]` — passwords (plaintext, не E2E)

Все мутации идут через `entriesApi.update(id, { body?, metadata? })`. Метаданные **всегда отправляются объектом целиком** (с merge), чтобы очистка поля реально очищала на сервере.

Если когда-то понадобится подобная панель для другой категории — паттерн копируется: новый компонент, условный рендер на entry detail page, поля в metadata. Это легче чем тащить per-category поля в саму схему `entries`.

18.7 Auto-translate via Google gtx

`lib/translate-client.ts` использует unofficial endpoint `translate.googleapis.com/translate_a/single?client=gtx`. Стабилен годами, CORS-friendly, без ключа. 5-секундный AbortController timeout. При фейле возвращает оригинал — translation там лучше английского, английский лучше пустоты.

Используется в:

- `AddItemModal` extraction effect — переводит `og:title` и `og:description` перед заполнением формы (если ≥ 30 % кириллицы — no-op)
- `VideoSummary` — постпроцессинг тезисов LLM-извлечения
- `translateArrayToRussianBrowser` — массовый параллельный перевод

18.8 SW versioning для force-reload

`public/sw.js` имеет `VERSION` в заголовочном комментарии (v2.x). При bump'е версии браузер видит новый файл и активирует SW; в `activate` event handler — `clients.matchAll()` + `navigate(c.url)` прогоняет force-reload по всем открытым вкладкам.

Используем когда:

- Меняется код, который пользователь точно должен подхватить (новые компоненты, фиксы багов)
- Service-worker fetch-стратегия меняется (passthrough vs cache)

Не нужно при обычных deploy'ях — Vercel CDN с правильными Cache-Control headers сам обновит чанки на следующей навигации.